CFPB Consumer Complaints Warehouse

Interview study guide — dbt + DuckDB + Power BI project, built by Yagas Vashist. Written to be read with zero prior dbt/analytics-engineering background: every number is traced back to how it was actually calculated, every command is explained, and one complaint is walked through the full pipeline as a worked example.

---

1. Project overview

This project turns CFPB's public Consumer Complaint Database — roughly 16.8 million real complaint records against banks, lenders, and credit reporting agencies — into a tested, documented analytics warehouse. dbt-core handles transformation and testing, DuckDB is the embedded engine, and Power BI sits on top as the presentation layer. The point of the project isn't the dashboard — it's proving the pipeline underneath the dashboard is correct, tested, and defensible.

Key numbers (final, verified run)

Metric	Value
Data snapshot date	2026-08-01
Raw rows ingested	16,849,715
Fact rows (final)	16,848,394
Rows excluded at staging	1,321
Distinct companies	8,033
Distinct product / sub-product combos	105
dbt models	7 (1 staging, 1 intermediate, 5 marts)
dbt tests	35 (34 pass, 1 warn, 0 error)
dbt total nodes (models + seed + tests)	43
dbt build runtime	63.23 seconds
Parquet export size (fact_complaints)	79.15 MB, ZSTD compressed

2. How the numbers reconcile — the actual arithmetic

This is the section to actually understand, not just memorize. Every count in this project traces back to one subtraction, done in two steps.

Step 1 — raw to fact

16,849,715 (raw rows loaded from the CFPB export, untouched)

– 1,321 (rows excluded by the staging layer — see step 2)

= 16,848,394 (rows in fact_complaints, the final count)

Step 2 — what makes up the 1,321 exclusions

11 (CSV multiline parse failures)

+ 1,310 (CFPB “Pending Company Match” records)

= 1,321 (total exclusions, matches step 1)

Why the 11 parse failures happen

CFPB's free-text complaint narrative field can legitimately contain a literal newline character — a consumer's complaint text can span multiple lines. A properly quoted CSV handles this fine (the whole field, newline included, sits inside one pair of quotes). If a row isn't quoted correctly in the source export, a naive parser reads that embedded newline as “end of row” and splits one real complaint into two broken rows — the second fragment has no ID, no date, nothing usable. That's what these 11 rows are: confirmed by inspection, not assumed.

Why the 1,310 Pending Company Match rows happen

A CFPB complaint goes through a lifecycle: submitted by the consumer, reviewed by CFPB, then routed to the company it's about. The Complaint ID is assigned as part of that routing step. These 1,310 rows are complaints that have cleared the first steps — they have a real date, a real state, a real product category — but haven't reached routing yet, so there's no ID. They are not bad data. They're excluded from fact_complaints for a structural reason, not a quality reason: the fact table's grain is “one row per complaint_id,” and a table can't enforce uniqueness or support reliable joins on a key that's null.

Analogy for a beginner: Think of a primary key like a tracking number on a package. If you have a box (the complaint) but FedEx hasn't printed the label yet (the null ID), you can't put it on the delivery truck (the fact table) because the system won't know how to track it. The box isn't broken, it's just not ready to ship.

Why this distinction matters for an interview

Anyone can say “I dropped 1,321 null rows.” The stronger answer is knowing that 11 of them are a genuine parsing defect and 1,310 of them are a completely different, non-defective category that just structurally can't fit a star schema yet. Conflating the two would be the naive-cleaning answer this project is specifically built to avoid.

3. Architecture walkthrough

Five stages, each independently re-runnable. The important thing to be able to explain isn't just what each stage does — it's why the boundary between stages is drawn where it is.

Stage	What it does	Why the boundary is there
1. Raw ingestion	Load the CFPB CSV unmodified into DuckDB, every column as VARCHAR.	Preserves source fidelity. If a downstream transformation turns out wrong, you rebuild from raw without re-downloading or re-trusting a cleaned copy.
2. Staging	1:1 with the source: rename to snake_case, cast dates and IDs. No business logic.	Separates structural cleanup (types, names) from business logic, so a broken cast is easy to isolate and never hides inside a business rule.
3. Intermediate	Dedup on complaint_id, standardize product labels across filing years, derive days_to_company_routing.	Keeps business logic in one auditable layer that can change without touching what the marts or the dashboard consume.
4. Marts	Star schema: fact_complaints plus dim_company, dim_product, dim_state, dim_date.	This is the shape BI tools actually query. A star schema keeps joins simple and query performance predictable at scale.
5. Presentation	Export marts to Parquet; Power BI reads Parquet directly.	Avoids a fragile ODBC driver dependency and keeps the project portable to any BI tool, not just Power BI.

4. Step-by-step workflow — what each command actually does

This is the same sequence as the README's setup instructions, with the “why” filled in for each step.

Command	What it does and why
python -m venv .venv / activate	Creates an isolated Python environment scoped to this project, so its exact package versions never conflict with anything else installed on the machine.
pip install -r requirements.txt	Installs dbt-core, the dbt-duckdb adapter, and the duckdb library — everything the pipeline needs, pinned to known-working versions.
python scripts/01_download.py	Downloads the ~330 MB zipped CFPB export from the government's public bulk URL and unzips it locally. Nothing is parsed or interpreted yet, just fetched.
python scripts/02_ingest.py	Creates the local DuckDB file and loads the raw CSV directly into a table called raw_complaints, with every column stored as text (VARCHAR). No cleaning, casting, or renaming happens at this step — it's a faithful copy of the source.
dbt seed --profiles-dir .	Loads the small us_states.csv lookup file (state code, state name, Census region) into DuckDB as a real table — this becomes the basis for dim_state.
dbt build --profiles-dir .	The core step. dbt reads every .sql file under models/, works out which models depend on which others via their ref() calls, runs them in the correct dependency order (staging, then intermediate, then marts), and after each model finishes, runs whatever tests are configured for it in that layer's schema.yml.

Command	What it does and why
dbt docs generate + dbt docs serve	Turns the project's model and column metadata into a browsable local website, including the interactive lineage graph (the DAG) showing exactly how every table depends on every other.
python scripts/04_export_parquet.py	Reads each finished mart table out of DuckDB and writes it to a .parquet file, which Power BI can open directly with no driver or database connection required.

5. A worked example — one complaint's journey through the pipeline

The table below is an illustrative example built to show the transformation clearly — not an actual extracted record. It follows one hypothetical complaint through all five layers.

Raw layer (all columns VARCHAR, no changes)

Column	Value
Complaint ID	"9284017"
Date received	"03/14/2024"
Product	"Credit Card or prepaid card"
Date sent to company	"03/19/2024"
Timely response?	"Yes"

Staging layer (renamed, cast, no business logic)

Column	Value
complaint_id (BIGINT)	9284017
date_received (DATE)	2024-03-14
product (VARCHAR, unchanged)	Credit Card or prepaid card
date_sent_to_company (DATE)	2024-03-19
timely_response (VARCHAR, unchanged)	Yes

Intermediate layer (business logic applied)

Column	Value
complaint_id	9284017
product_standardized	Credit Card / Prepaid Card (mapped from the raw label)
days_to_company_routing	5 (2024-03-19 minus 2024-03-14)
timely_response_flag (BOOLEAN)	TRUE

Marts layer (split into fact + dimension references)

Table	What it holds for this complaint
fact_complaints	complaint_id=9284017, product_id=<lookup>, date_received_key=<lookup>, days_to_company_routing=5, timely_response_flag=TRUE
dim_product	The row where product_name = ‘Credit Card / Prepaid Card’ — referenced by product_id, not repeated on every fact row
dim_date	The row where date_key corresponds to 2024-03-14, carrying year/month/quarter/day-of-week — also referenced, not repeated

This is the actual point of a star schema: the fact row stays narrow (just keys and measures), and everything descriptive — the full product name, the calendar attributes — lives once in a dimension table and gets referenced, not repeated 16.8 million times.

6. Core concepts — what you need to actually know, not just recite

dbt and the staging → intermediate → marts pattern

dbt (data build tool) turns SQL into a managed project: models are just SELECT statements in .sql files, and dbt figures out the run order automatically from ref() calls between them — that dependency graph is the DAG you see in dbt docs. Layering models into staging, intermediate, and marts isn't a dbt requirement, it's a convention that keeps each layer doing one job: staging cleans structure, intermediate applies business logic, marts is what gets queried.

Analogy for a beginner: Imagine SQL without dbt is like cooking a 5-course meal by memorizing every recipe and doing it all from scratch every time. dbt is like a smart recipe manager that says "chop the onions first, because the sauce needs them, and the pasta needs the sauce." It figures out the correct order to run everything.

Star schema, grain, fact vs. dimension

A fact table holds events or transactions at a defined **grain** — the grain answers "what does one row mean?" Here, the grain of fact_complaints is one row per complaint_id. Dimension tables hold descriptive context (company name, product name, state, date attributes) that the fact table references by key, as shown in the worked example above.

*Analogy for a beginner: A **Fact table** is like the printed receipt from a grocery store. It shows the transaction: time, barcode, quantity, total price. A **Dimension table** is the store's central product catalog. The receipt doesn't print the manufacturer's address and full nutritional info on every line; it just prints a barcode (the key). If you want those details, you look up the barcode in the catalog (the dimension).*

Why DuckDB

DuckDB is an embedded, in-process, columnar OLAP engine — no server, no setup, just a file. Columnar storage means it reads only the columns a query touches instead of whole rows, which is exactly the access pattern analytical aggregation needs. Row-oriented engines like Postgres or SQLite are built for OLTP workloads (many small reads and writes of individual rows), not for scanning and aggregating millions of rows at once.

Analogy for a beginner: A row-oriented database (like Postgres) stores data like a phone book. If you want to find John Smith's number, you read his whole row: name, address, phone. But if you want to find the average age of everyone in the book, you still have to read every name and address just to get to the ages. A columnar database (like DuckDB) stores all the ages together in one list, all the names in another. When you ask for the average age, it ignores the names and addresses entirely and just zips down the age list. That's why it's infinitely faster for analytics.

Materialization: why views here, tables there

Staging and intermediate models are materialized as **views** — they cost nothing to store and always reflect the latest model logic on every query. Marts are materialized as **tables** — they cost storage and a rebuild on every dbt run, but Power BI hits them repeatedly, so paying the computation cost once per build is the right trade rather than recomputing the whole transformation chain on every dashboard refresh.

dbt tests: types and severity

Test type	What it checks
not_null	The column never contains a null value.
unique	Every value in the column appears exactly once.

Test type	What it checks
relationships	Every value in this column exists in the referenced table's key column — referential integrity, i.e. a foreign-key check.
accepted_values	The column only ever contains values from a fixed, defined list.

Severity is a separate axis from the test type: **error** severity fails the whole dbt build; **warn** severity logs the failure but lets the build succeed. This project uses warn where the “violation” is real, legitimate data — military postal codes (AA/AE/AP) not in the state seed, for example — rather than an actual defect.

*Analogy for a beginner: An **Error** is a red traffic light. The data is fundamentally broken (like a missing primary key) and the pipeline must stop. A **Warn** is a yellow light. The data looks unusual (like a ZIP code from a military base abroad) so the system logs it to tell you to be careful, but lets the pipeline keep running.*

7. Design decisions and their defensible rationale

Every row here is a question an interviewer could reasonably ask — “why did you do X and not Y” — with the answer already worked out.

Decision	Why it's defensible
Parquet export, not DuckDB ODBC, for Power BI	DuckDB's Windows ODBC driver has known compatibility issues with Power BI Desktop. Parquet needs no driver install and is portable to any BI tool, not just Power BI.
profiles.yml committed to the repo	Contains only a local DuckDB file path, no credentials. Committed deliberately for one-command reproducibility on a solo/local project — explicitly acknowledged as non-standard for a team or production repo, where it would hold credentials and stay out of version control.
Field named days_to_company_routing, not “resolution time”	CFPB publishes only a routing date (when a complaint was forwarded to the company), not an actual resolution date. Naming the field for what the data really measures avoids overclaiming what the metric can support.
Warn, not error, severity on 3 specific tests	state_code relationship, timely_response not_null, and submitted_via accepted_values are all warn-severity because the underlying condition is real, valid data (military postal codes, in-progress complaints) — failing the entire build over legitimate data would be the wrong trade-off.
complaint_id cast to BIGINT, not INTEGER	No functional need at 16.8M rows today, but removes any doubt about integer overflow as the dataset grows — zero cost, permanently removes a class of bug.
Dedup on complaint_id, keep-latest by date_received	Keeps fact_complaints at a clean one-row-per-complaint grain, which every downstream join and every BI aggregate implicitly depends on.
Test-break injection uses a fabricated sentinel ID, not a real complaint_id	Duplicating a real ID meant cleanup had to guess which of two identical-looking rows to delete — that ambiguity is what caused a real row to be lost the first time. A fabricated ID that can never collide with real data makes cleanup unambiguous and safe to re-run.

8. The two real findings in the data

8a. Zero organic duplicate complaint IDs

Running the unique test against all 16.8 million real rows produced zero failures — bureau-assigned IDs are clean. That result alone doesn't prove the test works, though; it only proves the data happens to be clean. To actually verify the test catches what it's supposed to catch, two rows sharing a deliberately fabricated ID were injected, the test was confirmed to fail, and the injection was then removed. That's a stronger claim than “I wrote a test” — it's “I proved the test catches what it's built to catch.”

8b. CFPB “Pending Company Match” records

1,310 rows have a null complaint_id but otherwise valid dates, companies, and states — see section 2 for the full explanation of why. They're excluded from fact_complaints because the fact table's grain requires a primary key, and documented as a named category rather than silently dropped. This shows understanding of the source system's actual lifecycle, not just its file format.

9. The bug, and what it's actually worth in an interview

While verifying the unique test, the original injection script duplicated a real, existing complaint_id, then removed the duplicate as part of cleanup — but the cleanup logic wasn't scoped precisely enough, and it also deleted the original real row sharing that ID. One legitimate record was permanently lost from the raw table.

Root cause: the test mutated a shared, real table using an ID it didn't fully own — once two rows had the same ID, nothing distinguished “the one I added” from “the one that was already there.”

The fix: inject rows using a fabricated sentinel ID that is guaranteed not to collide with any real CFPB complaint_id. Cleanup then deletes only rows matching that exact sentinel value, which can never touch real data, because no real row will ever carry that ID.

How the original bug was found: the raw ingestion count (16,849,715) didn't reconcile with the current raw row count after rebuilding the fact table — the arithmetic was off by exactly one row, which is what surfaced the deletion. This is the same reconciliation discipline used in section 2 to explain every other number in this project.

Why this is a genuinely good interview answer: it's a real, specific, technical mistake with a clear root cause and a concrete fix — far stronger than a vague “I sometimes make mistakes but I learn from them.” Suggested structure for “tell me about a mistake you made”: what happened, why (an ID-based cleanup with no way to distinguish injected from real), how it was caught (a reconciliation check that didn't add up), and the fix (a sentinel ID that structurally cannot collide with real data).

10. Likely interview questions and talking points

Why dbt instead of plain SQL scripts?

dbt adds dependency management — it derives run order from ref() calls — plus built-in testing and auto-generated documentation and lineage. The same SQL without dbt would need all of that built by hand.

Why DuckDB instead of Postgres or SQLite?

DuckDB is columnar and in-process, built for scanning and aggregating large read-heavy datasets fast with zero setup. Postgres and SQLite are row-oriented, built for transactional workloads, not analytical scans across millions of rows.

What's the grain of your fact table?

One row per complaint_id. Every measure and every join assumes that grain, which is exactly why the uniqueness test on complaint_id in the fact table is non-negotiable.

Walk me through what happens when you run dbt build.

dbt reads the dependency graph built from ref() calls, determines run order, materializes each model as configured (view or table), then runs the tests configured against the results — failing or warning per each test's severity.

How did your raw and fact row counts reconcile?

16,849,715 raw rows minus 1,321 exclusions equals 16,848,394 fact rows. The 1,321 splits into 11 genuine CSV parse failures and 1,310 CFPB Pending Company Match records — see the arithmetic walkthrough in section 2 for the full reasoning behind each category.

What would you do differently in production?

Incremental loading instead of a full rebuild each run, an orchestrator like Airflow or Dagster instead of a manual setup script, real secrets management instead of a locally committed profiles.yml, and CI running dbt tests on every pull request before merge.

Tell me about a mistake you made on this project.

The original test-break cleanup script deleted a real row alongside the injected one, because it duplicated a real ID instead of using a fabricated one — see section 9 for the full structure of this answer.

Why isn't your routing-time field called “resolution time”?

CFPB doesn't publish an actual resolution date, only a routing date. Naming the field accurately avoids claiming a metric the data can't support.

How do you know your dedup logic actually works, if you found zero real duplicates?

Zero real duplicates only proves the data is clean, not that the test works. A duplicate was deliberately injected using a fabricated ID to confirm the test fails when it should, then removed safely.

What are the Pending Company Match rows, and why exclude them?

Complaints still in CFPB's internal routing queue, not yet assigned an ID — valid records, just structurally unable to join into a star schema that requires a primary key.

Why views for staging and intermediate, but tables for marts?

Views cost nothing to store and stay current automatically; marts are queried repeatedly by Power BI, so materializing them as tables trades a one-time build cost for much faster repeat queries.

What's a surrogate key, and where do you use one?

A generated key (for example, via ROW_NUMBER()) used when the source doesn't provide a natural unique identifier. company_id and product_id are surrogate keys generated in the marts layer, since the raw data only has names, not IDs.

Why warn instead of error on the state_code relationship test?

2,027 rows have state codes like AA, AE, or AP — real military postal codes not covered by the state seed. Failing the whole build over legitimate data would be the wrong trade-off; warn keeps the condition visible without blocking the pipeline.

11. Glossary

dbt (data build tool)
An open-source framework that turns SQL SELECT statements into a managed project with dependency resolution, testing, and documentation.

DAG
Directed acyclic graph — the dependency chain between models, derived automatically from ref() calls, with no circular dependencies allowed.

Materialization
How a dbt model is physically built: as a view (query run live each time) or a table (query run once, result stored).

Grain
The definition of what one row in a table represents — e.g. “one row per complaint.” Every measure and join in a model implicitly assumes its grain.

Star schema
A modeling pattern with one central fact table joined to surrounding dimension tables, shaped like a star when diagrammed.

Fact table
Holds events or transactions at a defined grain, referencing dimensions by key.

Dimension table
Holds descriptive context (names, categories, dates) that a fact table references, typically wide and slowly changing.

Surrogate key
An artificially generated identifier (e.g. via ROW_NUMBER()) used when the source system has no natural unique key.

Natural key
An identifier that already exists in the source data, such as CFPB's own complaint_id.

Sentinel value
A deliberately chosen value guaranteed not to occur in real data, used to mark or test something safely — the fabricated ID used in the fixed test-break script is a sentinel value.

Idempotent
A process that produces the same result no matter how many times it's re-run with the same input — important for pipelines that might be re-triggered.

Lineage
The traceable path of how a piece of data flowed from source to final output, visualized by dbt docs as the DAG.

ODBC
Open Database Connectivity — a standard driver interface that lets tools like Power BI connect to a database; DuckDB's ODBC support on Windows is not fully reliable.

Parquet
A columnar file format for storing tabular data efficiently and compactly, readable natively by most modern BI and data tools without a database connection.

Columnar storage
Data stored column-by-column rather than row-by-row, so a query only reads the columns it actually needs — ideal for analytical aggregation.

OLAP vs. OLTP
OLAP (online analytical processing) is optimized for large read-heavy aggregation queries; OLTP (online transactional processing) is optimized for many small, frequent reads and writes of individual records.

Seed
A dbt term for a small reference dataset (e.g. a state-code lookup CSV) loaded into the warehouse directly from a file in the repo, via dbt seed.

ref() function
The dbt macro used inside a model to reference another model, which is how dbt builds the dependency graph automatically.

12. Known limitations — know these before someone else points them out

- Full-snapshot load, not incremental — every run rebuilds from the complete CFPB export. Fine at this scale and frequency; a true production pipeline updating daily or hourly would use dbt's incremental materialization instead.
- dim_state's seed doesn't cover military postal codes (AA, AE, AP) — acknowledged explicitly via the warn-severity relationship test rather than silently ignored.
- No CI/CD — tests currently run manually via dbt build, not gated automatically on every change to the repo.
- No orchestration tool — the manual setup.ps1 script stands in for what would be an Airflow or Dagster DAG in a real production analog.

Before the interview, actually do this

- Re-run scripts/03_inject_test_break.py end to end and confirm raw count returns to 16,849,715 unchanged afterward — that's the real proof the fix works, not the fact that the number looks right today.
- Open dbt docs and click through the lineage graph once more so you can describe it without looking at this guide.
- Re-run the sample query in the README from memory, out loud, once.

Study approach: read sections 2, 4, and 5 first — those build the actual understanding. Section 10 is what interview questions will look like; section 9 is the strongest single story in this whole document. Know it well enough to tell it without notes.